

A Crash Course in Arduino Programming

Lesson 3

A. Introduction

Welcome to lesson 3! Today we are talking about, arrays, bitwise operations, and two common communication protocols.



B. Arrays

According to the official Arduino Docs, an **array** is a collection of variables that are accessed with an indexed number. You can think of it like a matrix with 1 row and many columns. To use arrays, you'll have to know how to create them and how to **access** them.

To access an array, we use the following syntax¹:

```
myArray[element_index];
```

`myArray` is the name of the variable, and `element_index` is an integer number that tells the program which element of the array we want to look at. Suppose the contents of the array are {1, 2, 3}. If we wanted to grab the first element of the array (which is 1 in this case), what would `element_index` need to be? Using an index of 1 would be a reasonable guess², but we would actually use an index of 0. So, we use 0 to access the first element, 1 to access the 2nd element, and 2 to access the 3rd element.

There are several different ways to make an array, so let's go over each method using an integer array holding the values 1, 2, and 3.

Method 1

```
// Method 1: Declare an array of a given length without
// initializing any of the values
int myInts[3];
myInts[0] = 1;
myInts[1] = 2;
myInts[2] = 3;
```

¹ The syntax for accessing an array is similar to accessing a list in Python! Additionally, the way we use arrays in Arduino is similar to how we would use a list in Python.

² I am aware of you, MATLAB users....

Method 2

```
// Method 2: Declare an array of a given length without
// initializing any of the values, but this time its a for loop
int myInts[3];

for (int ii=0; ii<3; ii++){
    myInts[ii] = (ii+1);
}
```

Method 3

```
// Method 3: We don't explicitly specify the length, but
// the program knows the length because we provide the values
// of the array.
int myInts[] = {1, 2, 3};
```

Method 4

```
// Method 4: Explicitly specify everything
int myInts[3] = {1, 2, 3};
```

C. Bits

A binary number is made of bits (digits that can take on a value of either 1 or 0). In Arduino, the numbers 1 and 0 can also represent true and false. Think back to lesson when we covered the concepts of OR and AND. We can use this concept to combine entire binary numbers! Let's illustrate with an example.

Let's say we have two binary numbers:

A = 0101

B = 1010

A OR B = 1111

A	0	1	0	1
B	1	0	1	0
A OR B	1	1	1	1

A AND B = 0000

A	0	1	0	1
B	1	0	1	0
A AND B	0	0	0	0

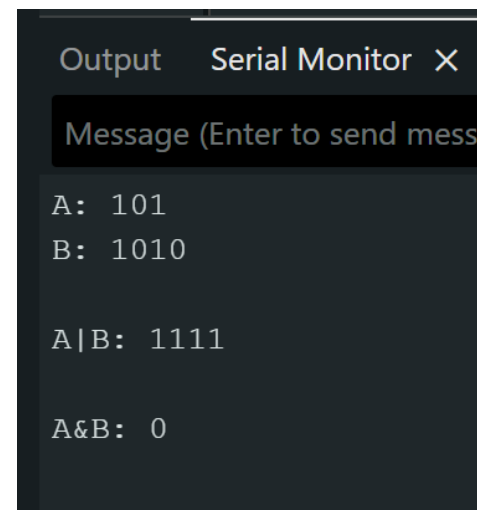
In Arduino, we can do these same operations with **Bitwise AND** and **Bitwise OR**.

```
void setup() {
  Serial.begin(9600);
  delay(1000);

  byte A = 0b0101;
  byte B = 0b1010;

  Serial.print("A: "); Serial.println(A, BIN);
  Serial.print("B: "); Serial.println(B, BIN);

  Serial.print("\nA|B: "); Serial.println(A|B, BIN);
  Serial.print("\nA&B: "); Serial.println(A&B, BIN);
}
```



What if instead of combining a number into a different 4 bit number, we want to stick the numbers back to back to make an 8 bit number? Before we combine our numbers, one of them has to be **shifted**.

This is A (0101)

0	1	0	1
---	---	---	---

We can shift A four spaces to the left with the operator `<<`. This is `(A<<4)`.

0	1	0	1	0	0	0	0
---	---	---	---	---	---	---	---

Now, we can stick B onto the end of this number using bitwise OR.

A	0	1	0	1	0	0	0	0
B	0	0	0	0	1	0	1	0
A B	0	1	0	1	1	0	1	0

```

byte A = 0b0101;
byte B = 0b1010;

Serial.print("A: "); Serial.println(A, BIN);
Serial.print("B: "); Serial.println(B, BIN);

Serial.print("\nA<<4: "); Serial.println(A<<4, BIN);
Serial.print("\n(A<<4)|B: "); Serial.println((A<<4)|B, BIN);

```

```

A: 101
B: 1010

A<<4: 1010000
(A<<4)|B: 1011010

```

D. Serial-Peripheral Interface (SPI)

SPI is a **communication protocol**; it let's us send information between the Arduino and one or more **peripheral devices**. Compared to other common communication protocols, SPI is fast. The terminology used to refer to the devices involved and the direction of information flow varies. Here's a table summarizing the terms you'll probably come across.

Old Terminology	New Terminology	Forest's Terminology
Master	Controller	Central
Slave	Peripheral	Peripheral
Master In Slave Out (MISO)	Controller In Peripheral Out (CIPO)	Central In Peripheral Out (CIPO); Afferent
Master Out Slave In (MOSI)	Controller Out Peripheral In (COPI)	Central Out Peripheral In (COPI); Efferent
Slave Select (SS)	Chip Select (CS)	Chip Select (CS)

I refer to the devices as either central or peripheral, because I am making an analogy between SPI and the nervous system³. The central device (the microcontroller) is the brains of the operation. It takes in all of the information from the peripheral sensors, interprets it, then sends out signals that tell the peripheral devices what to do. Your network of peripheral devices is kind of like your peripheral nervous system. The sensors take in information about the environment, then they report back to central so the information can be processed and decisions can be made.

Central In Peripheral Out (CIPO) is the line used by the peripheral device to send data to the central device. **Central Out Peripheral In (COPI)** In is the line used by the central device to send data to the peripherals. There is a **clock (SCK) line** which pulses in order to synchronize data transmissions generated by the central device. The **chip select (CS)** line is used by the central device to enable or disable peripheral devices. When a device's CS pin is low, it communicates with the central device. When the CS pin is high, it ignores the central device. This allows one central device to connect to many peripheral devices while using the same CIPO, COPI, and SCK lines.

E. Inter-Integrated Circuit (I2C)

I2C is another common communication protocol. Like SPI, I2C has one central device and can have many peripheral devices. However, I2C does not have separate lines for communication from central and communication from the peripherals. Instead, the central device and the peripheral devices have to take turns sending information on *the same data line*. This is one of the reasons why I2C is typically slower than SPI.

I2C utilizes two wires, one for data, **SDA**, and another for the clock, **SCL**⁴. Since the devices share a data line, whenever we want to change the direction of information flow, we have to do something called a **repeated start**. Another difference between I2C and SPI is how they differentiate between peripheral devices. SPI uses a chip select line, but I2C doesn't have a wire for that. Instead, each I2C device has its own **register**. This register is a 7-bit number, typically represented in **hexadecimal**. The central device begins each transmission by specifying the register of the peripheral device it would like to communicate with.

Whatever I2C device you're working with, whether you're on an Uno, a Nano, or a Pi, reading and writing is going to follow the same general structure.

Writing to a peripheral device:

1. Begin transmission to the device using the device register
2. Write the register of the device you would like to access
3. Write whatever information you want to send to that register
4. End transmission

³ Ha! Nerd.

⁴ Some people like to use specific colors for SDA and SCL in their circuits. I'm partial to using yellow for SDA and blue for SCL.

Reading from a peripheral device:

1. Begin transmission to the device using the device register
2. Write the register of the device you would like to access
3. End transmission (repeated start to change information direction)
4. Request bytes from the peripheral device
5. Read bytes
6. End transmission

This is an example of reading data from a sensor looks like. The yellow (top) line is SDA and the red (bottom) line is the clock.

